

1 P2 — Uphill paths on hypercubes: C1–C4

This writeup supplies the values and complete binary vertex lists required by [the P2 task](#). The page was audited on 2026-09-26: C1–C4 request a value and an explicit list of **every** vertex in increasing label order, and the cells are marked correct on the values alone. The lists below satisfy that format. They are submission materials, not a claim that a platform submission was made. C5 and C6, concerning Q_9 , were intentionally skipped.

Cell	Dimensions	$U(Q_d)$	Task status	Lean optimality coverage
C1	3, 4	14, 34	Solved	Universal lower bounds and attaining lists proved in Lean.
C2	5	88	Solved	Attaining list proved; universal lower bound has explicit unproved Lean premises, with independent finite replays below.
C3	6	204	Solved	Attaining list proved; face-doubling lower bound is written below.
C4	7, 8	464, 1040	Solved	Attaining lists proved; face-doubling lower bounds are written below.
C5	9	—	Skipped	Bound-improvement work paused.
C6	9	—	Skipped	Open exact-value work paused.

For Q_d , vertices are width- d binary strings. Two vertices are adjacent when they differ in one bit. A labelling bijectively assigns $1, \dots, 2^d$ to the vertices. A valley has no lower-labelled neighbour. An uphill path starts at a valley and follows edges with strictly increasing labels; a singleton valley counts as one path. Write $P(\lambda)$ for the path count of labelling λ , and $U(Q_d) = \min_{\lambda} P(\lambda)$. In each list below, the **first line has label 1**, the next line label 2, and so on; the lists are complete permutations, not just the vertices on one path.

1.1 Counting paths and reducing to decycling

Orient each edge from lower to higher label. Let $p(v)$ be the number of uphill paths ending at v . Processing vertices in label order gives the exact recurrence

$$p(v) = \begin{cases} 1, & v \text{ is a valley,} \\ \sum_{u \rightarrow v} p(u), & \text{otherwise.} \end{cases}$$

In particular $p(v) \geq 1$. Define $B = \{v : p(v) > 1\}$ and write $b = |B|$. Every cycle meets B : the highest-labelled vertex of a cycle has two lower-labelled cycle neighbours, so its endpoint count is at least two. Thus B is a decycling (feedback vertex) set, and $b \geq \nabla(G)$, the minimum number of vertices whose removal leaves a forest.

Here is the more precise path budget for any d -regular graph G with n vertices and m edges. Write s for the number of valleys and e_B for edges with both ends in B . An edge cannot point from B to its complement: the endpoint would then receive at least two paths. Every non-valley outside B has exactly one incoming edge, also from outside B . Counting these $n - b - s$ edges and the $db - e_B$ edges incident to B yields

$$m = (n - b - s) + db - e_B, \quad s = n - m + (d - 1)b - e_B.$$

Count each non-singleton path by its last oriented edge. Since each endpoint count contributes $p(u)$ paths to every outgoing edge,

$$\begin{aligned} P(\lambda) &= s + \sum_{u \rightarrow v} p(u) \\ &= m + s + \sum_{u \in B} (p(u) - 1) \text{outdeg}(u) \\ &\geq m + s + e_B \\ &= n + (d - 1)b \\ &\geq n + (d - 1)\nabla(G). \end{aligned}$$

The third line holds because each internal B -edge contributes at least one through its lower-labelled end. The recurrence, endpoint decomposition, and budget are formalized in the [Lean hypercube and lower-bound modules](#). The [exact path checker](#) independently evaluates this recurrence on each list.

1.2 C1: Q_3 and Q_4

There is at least one valley, and $e_B \geq 0$. The preceding identity for s therefore implies $(d - 1)b \geq m - n + 1$. For Q_3 , $(n, m, d) = (8, 12, 3)$, so $b \geq 3$ and $P(\lambda) \geq 8 + 2 \cdot 3 = 14$ for every labelling. For Q_4 , $(n, m, d) = (16, 32, 4)$, so $b \geq 6$ and $P(\lambda) \geq 16 + 3 \cdot 6 = 34$. The following lists attain those bounds. The packaged [C1Optimal.lean](#) proves both universal optimality theorems, `q3_optimal` and `q4_optimal`, from the attaining [Q3](#) and [Q4](#) certificates. The [C1 verification log](#) records a build, leanchecker replay, and axiom audits.

C1, Q_3 , value 14. Complete [source list](#):

```
000
001
```

010
100
111
011
101
110

C1, Q_4 , value 34. Complete [source list](#):

0000
1111
0001
0010
0100
0111
1000
1011
1101
1110
0011
0101
0110
1001
1010
1100

1.3 C2: Q_5

The [Q5 witness certificate](#) proves that the complete list below has 88 paths. To prove no labelling has fewer paths, it suffices by the budget to show $\nabla(Q_5) \geq 14$. The following is an explicit, finite computational proof with two independently replayable algorithms; it is **not** a Lean proof of that final inequality.

Suppose a deletion set S of at most 13 vertices leaves a forest. Extend it to 13 vertices; extra deletions preserve the forest property. Let $e(S)$ count edges inside S . The 19-vertex complement has $80 - 5 \cdot 13 + e(S) = 15 + e(S)$ edges. A forest with 19 vertices has at most 18 edges, so necessarily $e(S) \leq 3$.

Split the vertices into 16 even- and 16 odd-Hamming-weight vertices; every cube edge crosses the split. XOR with the low bit swaps parity while preserving adjacency. Thus we may choose the side with fewer selected vertices as the even side, containing $k \leq 6$ vertices. Fix that even subset A . Each odd vertex v has cost $|N(v) \cap A|$, and the internal-edge count of a selected set with $13 - k$ odd vertices is the sum of their costs. The minimum possible sum is therefore the sum of the $13 - k$ smallest of the 16 costs. The [parity/forest replay](#) enumerates every A and obtains:

k	3	4	5	6
Even subsets examined	560	1,820	4,368	8,008
Minimum possible $e(S)$	4	4	5	5

Hence $e(S) \leq 3$ requires $k = 0, 1$, or 2 . For each of these cases, the replay enumerates the required odd subsets by include/exclude recursion. It discards a branch only when its accumulated cost already exceeds 3 or too few unvisited odd vertices remain to fill the set. This examines respectively 560, 10,640, and 1,600 affordable 13-vertex sets, or 12,800 in all. For each, it independently generates cube edges by bit flips and uses union-find on the 19-vertex complement.

Every complement has a cycle. The [recorded run](#) gives the counts and a SHA-256 hash of the checked candidate stream. The pruning preserves every possible forest complement, so the finite cases exclude all $|S| \leq 13$.

As a separate cross-check, the [cycle-branch replay](#) generates all 80 cube edges, 80 squares, and 640 six-cycles. Any decycling set must hit each listed cycle. On the first unhit cycle, it branches on every vertex; it prunes only when more than 13 vertices are selected or the internal-edge count exceeds 3. That count can only increase as vertices are added, so both pruning rules are sound. Memoization keys the complete selected set. Its [recorded run](#) reports UNSAT, 42,324 states, and 61,072 branches. Neither search uses sampling or an external solver; both replay in well under ten minutes on the recorded machine. Thus $\nabla(Q_5) \geq 14$, and the path budget yields $P(\lambda) \geq 32 + 4 \cdot 14 = 88$.

C2, Q_5 , value 88. Complete [source list](#):

```
00000
01111
00001
00010
00100
00111
01000
01011
01101
01110
10000
10011
10101
10110
11001
11010
11100
11111
00011
00101
00110
01001
01010
01100
10001
10010
10100
10111
11000
11011
11101
11110
```

1.4 C3–C4: Q_6 , Q_7 , and Q_8

The two disjoint $(d - 1)$ -dimensional faces of Q_d are copies of Q_{d-1} . If S decycles Q_d , then its intersection with each face decycles that face. Their vertex sets are disjoint, so

$$\nabla(Q_d) \geq 2\nabla(Q_{d-1}).$$

Starting from $\nabla(Q_5) \geq 14$, this gives $\nabla(Q_6) \geq 28$, $\nabla(Q_7) \geq 56$, and $\nabla(Q_8) \geq 112$. Taking the minimum over labellings in the path budget gives

$$\begin{aligned}
U(Q_6) &\geq 64 + 5 \cdot 28 = 204, \\
U(Q_7) &\geq 128 + 6 \cdot 56 = 464, \\
U(Q_8) &\geq 256 + 7 \cdot 112 = 1040.
\end{aligned}$$

The following attaining lists are verified by [Q6.lean](#), [Q7.lean](#), and [Q8.lean](#). Their written lower bounds above complete the mathematical argument. Face doubling itself is not formalized in the packaged Lean sources.

C3, Q_6 , value 204. Complete [source list](#):

```

000000
001111
110011
111100
000001
000010
000100
000111
001000
001011
001101
001110
010000
010011
010101
010110
011001
011010
011100
011111
100000
100011
100101
100110
101001
101010
101100
101111
110001
110010
110100
110111
111000
111011
111101
111110
000011
000101
000110
001001
001010
001100
010001
010010
010100
010111
011000
011011
011101
011110
100001

```

100010
100100
100111
101000
101011
101101
101110
110000
110101
110110
111001
111010
111111

C4, Q_7 , value 464. Complete [source list](#):

0000000
0001111
0110011
0111100
1010101
1011010
1100110
1101001
0000001
0000010
0000100
0000111
0001000
0001011
0001101
0001110
0010000
0010011
0010101
0010110
0011001
0011010
0011100
0011111
0100000
0100011
0100101
0100110
0101001
0101010
0101100
0101111
0110001
0110010
0110100
0110111
0111000
0111011
0111101
0111110
1000000
1000011
1000101
1000110
1001001
1001010

1001100
1001111
1010001
1010010
1010100
1010111
1011000
1011011
1011101
1011110
1100001
1100010
1100100
1100111
1101000
1101011
1101101
1101110
1110000
1110011
1110101
1110110
1111001
1111010
1111100
1111111
0000011
0000101
0000110
0001001
0001010
0001100
0010001
0010010
0010100
0010111
0011000
0011011
0011101
0011110
0100001
0100010
0100100
0100111
0101000
0101011
0101101
0101110
0110000
0110101
0110110
0111001
0111010
0111111
1000001
1000010
1000100
1000111
1001000
1001011
1001101
1001110
1010000

1010011
1010110
1011001
1011100
1011111
1100000
1100011
1100101
1101010
1101100
1101111
1110001
1110010
1110100
1110111
1111000
1111011
1111101
1111110

C4, Q_8 , value 1040. Complete [source list](#):

00000000
00001111
00110011
00111100
01010101
01011010
01100110
01101001
10010110
10011001
10100101
10101010
11000011
11001100
11110000
11111111
00000001
00000010
00000100
00000111
00001000
00001011
00001101
00001110
00010000
00010011
00010101
00010110
00011001
00011010
00011100
00011111
00100000
00100011
00100101
00100110
00101001
00101010
00101100
00101111

00110001
00110010
00110100
00110111
00111000
00111011
00111101
00111110
01000000
01000011
01000101
01000110
01001001
01001010
01001100
01001111
01010001
01010010
01010100
01010111
01011000
01011011
01011101
01011110
01100001
01100010
01100100
01100111
01101000
01101011
01101101
01101110
01110000
01110011
01110101
01110110
01111001
01111010
01111100
01111111
10000000
10000011
10000101
10000110
10001001
10001010
10001100
10001111
10010001
10010010
10010100
10010111
10011000
10011011
10011101
10011110
10100001
10100010
10100100
10100111
10101000
10101011
10101101

10101110
10110000
10110011
10110101
10110110
10111001
10111010
10111100
10111111
11000001
11000010
11000100
11000111
11001000
11001011
11001101
11001110
11010000
11010011
11010101
11010110
11011001
11011010
11011100
11011111
11100000
11100011
11100101
11100110
11101001
11101010
11101100
11101111
11110001
11110010
11110100
11110111
11111000
11111011
11111101
11111110
00000011
00000101
00000110
00001001
00001010
00001100
00010001
00010010
00010100
00010111
00011000
00011011
00011101
00011110
00100001
00100010
00100100
00100111
00101000
00101011
00101101
00101110

00110000
00110101
00110110
00111001
00111010
00111111
01000001
01000010
01000100
01000111
01001000
01001011
01001101
01001110
01010000
01010011
01010110
01011001
01011100
01011111
01100000
01100011
01100101
01101010
01101100
01101111
01110001
01110010
01110100
01110111
01111000
01111011
01111101
01111110
10000001
10000010
10000100
10000111
10001000
10001011
10001101
10001110
10010000
10010011
10010101
10011010
10011100
10011111
10100000
10100011
10100110
10101001
10101100
10101111
10110001
10110010
10110100
10110111
10111000
10111011
10111101
10111110
11000000

```

11000101
11000110
11001001
11001010
11001111
11010001
11010010
11010100
11010111
11011000
11011011
11011101
11011110
11100001
11100010
11100100
11100111
11101000
11101011
11101101
11101110
11110011
11110101
11110110
11111001
11111010
11111100

```

1.5 Reproduction, references, and proof boundary

From the [package directory](#), `python3 shared/witnesses.py --check` compares all generated label and Lean witness files with the packaged versions. The two Q5 checks run as `python3 c2/decycling5_parity_replay.py` and `python3 c2/decycling5_replay.py`. The [original six-witness verification log](#), [C1 optimality log](#), and [incremental Q5 Lean log](#) document the builds and axiom checks. The packaged [verify.sh](#) gives the full replay sequence.

The Lean witness files prove each displayed list is a permutation and has its stated path count. C1 additionally has full Lean universal optimality. For C2, [Decycling5.lean](#) validates the listed cycles and conditional search soundness, while [DecyclingBridge.lean](#) proves the path budget and a conditional Q5 lower theorem. Its premises `Decycling5.search 13 [] = true` and `Decycling5.internalEdges B ≤ edgeCount 5 B` have not been discharged in Lean. The Python finite checks and the written face-doubling argument are independently inspectable evidence, but are not kernel certificates. The recorded `#print axioms` results for the proved Lean theorems use only `propext`, `Classical.choice`, and `Quot.sound`.

The exact hypercube decycling values 3, 6, 14, 28, 56, 112 for dimensions 3–8 were reported by Beineke and Vandell, *Decycling graphs*, *Journal of Graph Theory* 25 (1997), 59–77. The values appear in the primary-author [Bau–Beineke survey](#), §2, p. 288, [Zou’s thesis](#), §2.2.3, and [Hertz’s 2021 Table 4](#). Those reports corroborate the values; the parity and cycle-branch replays above reconstruct the finite Q5 obstruction independently. No cited result is imported as a Lean axiom.